Ferdowsi University of Mashhad

Department of Computer Engineering

Parity Search Engine

Operating Systems – Project 1

Technical Architecture & Implementation Report

Developer: Abolfazl Amani

Date: June 19, 2026

Contents

1. Introduction & Architecture

The **Parity Search Engine** is an advanced, concurrent, and fault-tolerant system engineered to parse and analyze massive log directories. The architecture is deeply rooted in POSIX standards, distributing the workload across three distinct operational layers to maximize CPU utilization and prevent blocking.

Architectural Layers

- **Layer 1 (Multi-Processing):** The *Manager* forks multiple *Searchers*. It enforces fault-tolerance via `waitpid` and aggregates child results using an IPC `pipe`.
- **Layer 2 (Multi-Threading):** Inside each Searcher, a dynamic Thread Pool operates on a Shared Work Queue, preventing I/O bottlenecks and ensuring overlapping disk reads.
- **Layer 3 (Synchronization):** Strict thread-safety is guaranteed using `pthread_mutex_t` for memory protection and `pthread_cond_t` for intelligent thread suspension (Zero Busy-Waiting).

2. Core Implementation Strategies

2.1 Smart Directory Traversal & Distribution

To achieve absolute parallelism, the system uses a recursive directory traversal algorithm coupled with a **Round-Robin Modulo** distribution strategy. Instead of dumping all files into one global bottlenecked queue, each Searcher process builds its own localized queue and claims ownership of specific files where:

$$File\_Index \pmod{Total\_Processes} == Process\_ID$$

This guarantees a perfectly balanced workload without requiring complex inter-process memory sharing.

2.2 Thread-Safe Work Queue & Deadlock Prevention

Concurrency Mechanics

The Work Queue is a robust, dynamic Linked List operating as the heart of the Thread Pool.

- **Enqueue:** Locks the queue → Inserts Node → Signals waiting threads via `pthread_cond_signal` → Unlocks.
- **Dequeue:** Locks → Suspends via `pthread_cond_wait` if empty → Extracts Node → Unlocks.

Important Architecture Note

Graceful Shutdown: Deadlocks are prevented during shutdown by injecting `"STOP"` poison pills into the queues. Threads consume these pills, release their dynamically allocated memory, and terminate safely without sleeping infinitely.

3. Process Monitor (Fault Tolerance)

The system is designed to survive unexpected worker crashes (e.g., Segmentation Faults triggered by corrupt files). The Manager operates an active supervision loop:

1. Monitors children via `waitpid(-1, &status, 0)`.
2. Analyzes the termination signal (`WIFEXITED` vs `WIFSIGNALED`).
3. If an abnormal crash is detected, the Manager identifies the exact partition of the crashed Searcher and immediately **forks a replacement process** to retry the workload, ensuring complete data processing reliability.

4. Performance Benchmarks

The system was benchmarked using the Linux `time` utility over a dataset of log files. The results illustrate the impact of optimal thread-to-process ratios.

Processes (`-p`)	Threads (`-t`)	Real Time	User Time	Sys Time
1	1	1m 12.0s	0m 45.2s	0m 20.1s
2	2	0m 40.5s	0m 48.3s	0m 22.4s
4	4	**0m 22.1s**	0m 52.1s	0m 26.5s
8	8	0m 20.8s	0m 55.4s	0m 30.2s

Table 1: Benchmarking results demonstrating concurrency speedup.

4.1 Benchmark Analysis

As observed, scaling parallel units drastically reduces *Real Time*. However, increasing the configuration beyond the hardware's physical cores (e.g., $P = 8, T = 8$) yields diminishing returns. Text searching is an I/O bound task; once the disk's read bandwidth is saturated, adding more threads only inflates Context-Switching overhead within the OS Kernel (visible in the increased *Sys Time*).